❸ The R2DBC URL identifying the database you want to establish a connection with

❹ The maximum time to wait to get a connection from the pool

❺ The initial size of the connection pool

❻ The maximum number of connections kept in the pool

Now that you have connected a reactive Spring Boot application to a PostgreSQL database through an R2DBC driver, you can go ahead and define the data you want to persist.

DEFINING PERSISTENT ENTITIES

The Order Service application provides functionality for submitting and retrieving orders. That's the *domain entity*. Add a new `com.polarbookshop.orderservice.order .domain` package for the business logic, and create an `Order` Java record to represent the domain entity, much like you defined `Book` in Catalog Service.

Following the same approach used in chapter 5, use the `@Id` annotation to mark the field representing the primary key in the database, and use `@Version` to provide a version number, which is essential for handling concurrent updates and using optimistic locking. You can also add the necessary fields to hold audit metadata using the `@CreatedDate` and `@LastModifiedDate` annotations.

The default strategy for mapping an entity to a relational table is to transform the Java object name into lowercase. In this example, Spring Data would try to map the `Order` record to an `order` table. The problem is that `order` is a reserved word in SQL. It's not recommended that you use it as a table name because it would require special handling. You can overcome that issue by naming the table `orders` and configuring the